



United International University

Department of Computer Science and Engineering

CSI 309/CSE 4509: Operating System Concepts/Operating Systems

Midterm Examination : Spring 2026

Total Marks: 30 Time: 1 Hour 30 Minutes

Answer all the 3 questions. Numbers to the right of the questions denote their marks.

Any examinee found adopting unfair means will be expelled from the trimester / program as per UIU disciplinary rules.

1. (a) i. What is a process in an operating system? [1]
ii. How does a process transition happen from the **embryo (new) state** to the **ready state**? [1]
- (b) A hospital management system loads patient records on startup. The system contains 50,000 patient records, but on any given day only the records of patients with scheduled appointments (typically 40–80) are accessed. The application currently takes 35 seconds to start because it pre-loads all records into memory.
 - i. Which loading strategy is the system currently using: **eager loading** or **lazy loading**? [1]
 - ii. Suggest a better alternative and explain how it would improve performance in this scenario. [1]
- (c) Consider the following C program:

```
int val = 5;

int main() {
    if (fork() > 0) {
        val += 10;
        if (fork() == 0) {
            val *= 2;
            printf("X: %d\n", val);
            exit(0);
        }
        wait(NULL);
        printf("Y: %d\n", val);
    } else {
        wait(NULL);
        printf("Z: %d\n", val);
    }
    wait(NULL);
    return 0;
}
```

- i. Draw the **process tree** generated by the given code. Label each node with the value of **val** at the moment it executes its corresponding **printf** statement in that process. [4]
 - ii. Find **one possible output sequence** produced by this program. [2]
2. Consider the following five processes arriving with the given burst times. The system uses **Round Robin (RR)** scheduling with a **time quantum of 4 ms**. Newly arrived processes are inserted into the ready queue **in order of arrival**. If processes arrive at the same time, ties are broken by **placing the lower burst time job first**. When the running process finishes, or its quantum expires, the ready queue is checked for new arrivals before the next process is selected.

Process	Arrival Time (ms)	Burst Time (ms)
P1	0	9
P2	0	3
P3	5	7
P4	10	4
P5	12	6

- (a) Draw the **Gantt chart** for the Round Robin schedule. Show the **state of the ready queue** at each scheduling decision point. [5]
- (b) Calculate the **Turnaround Time (TAT)** and **Response Time (RT)** for each process. Present your results in a table and compute the **average TAT** and **average RT**. [3]

- (c) A modified policy called **Priority-Aged Round Robin (PARR)** works as follows:
- Each process is assigned an initial priority equal to its burst time (lower value = higher priority).
 - The ready queue is always sorted by priority (ascending); ties are broken by earlier arrival time.
 - After every quantum, an unfinished process has its priority decreased by 2 (it “ages” and becomes more urgent), then re-enters the sorted queue.
 - New arrivals join the sorted queue immediately.

Using the same processes and quantum, would PARR give a **better or worse** average Turnaround Time compared to RR? **Justify your answer** with reasoning. You do *not* need to simulate PARR fully. [2]

3. (a) Write the differences between *Compare-and-Swap* lock and *Test-and-Set* spin lock. [2]
- (b) Can we achieve mutual exclusion using a Test-and-Set spin lock? Justify your answer using two processes. [3]
- (c) At the UIU canteen, there is a token-based system for accessing meals during peak hours. The token dispenser can hold up to N tokens at a time. Each student must collect one token before joining the food queue. If no tokens are available, students must wait. A staff member issues N tokens at once, but only when all tokens have been used. After issuing tokens, all waiting students should be notified.

The shared variables and the synchronized code for Student thread are given below. Now write the **synchronized version of the Staff thread** using **mutex locks** and **condition variables**. [5]

Shared Variables:

```
int tokens = 0; //number of tokens available
mutex m;
condvar noTokens, tokensAvailable;
```

Student Thread (Synchronized):

```
while(true) {
    lock(m);
    while(tokens == 0) {
        wait(tokensAvailable, m);
    }
    collectToken();
    tokens--;
    if(tokens == 0) {
        signal(noTokens);
    }
    unlock(m);

    goToFoodCounterAndEat();
}
```

Staff Thread (Unsynchronized):

```
while(true) {
    prepareTokens();
    issueTokens(N);
}
```